



Xadrez em Portugal: lógica de programação de forma lúdica

Relatório completo de projeto · Instituto Federal do Pará (IFPA) · Engenharia de Alimentos · Algoritmo e Lógica de Programação · Professor José Alcimar

Autores: Allan Rasec De Araujo Brito e Thiago Oliveira Da Silva · Outubro de 2026

Resumo

Este relatório apresenta um jogo de xadrez completo desenvolvido em Portugal, a linguagem de apoio ao ensino de lógica de programação. Ele mostra como o projeto foi construído em dez etapas, explica cada uma das 31 funções com a imagem do código e as decisões por trás dela, propõe uma metodologia de ensino chamada Jogada a Jogada e discute o que o projeto pode significar para alunos e professores.

Sumário

- | | |
|---|----------------------------------|
| 1. O projeto, abertamente | Etapa 6 · Xeque e xeque-mate |
| 2. Como isso pode mudar vidas | Etapa 7 · Recursos da partida |
| 3. Fundamentação | Etapa 8 · Acabamento e interface |
| 4. Metodologia Jogada a Jogada | Etapa 9 · Testes e versões |
| 5. Etapas e funções | Etapa 10 · Aplicação em sala |
| Etapa 1 · Ideia, público e planejamento | 6. Decisões e alternativas |
| Etapa 2 · Modelagem dos dados | 7. Plano de testes |
| Etapa 3 · Tela e layout | 8. Limitações |
| Etapa 4 · Interação com o mouse | 9. Conclusão |
| Etapa 5 · Regras de movimento | |

1. O projeto, abertamente

O que fizemos é um jogo de xadrez completo para dois jogadores no mesmo computador, escrito em Portugal, com cerca de 900 linhas e 31 funções. Ele tem tabuleiro gráfico, arrastar peças com o mouse, xeque, xeque-mate, voltar jogada, empate, desistência, tela cheia e som.

Fizemos porque a lógica de programação costuma chegar aos alunos como algo distante, principalmente em cursos que não são de computação, como a Engenharia de Alimentos. Queríamos mostrar o contrário: o Portugal, uma linguagem criada só para aprender lógica, também cria jogos.

Também queremos ser claros sobre o que o projeto não é. Ele não é um motor de xadrez com inteligência artificial e não cobre todas as regras oficiais. Foi feito por aprendizes de programação, tem limitações que listamos na seção 8 e é, antes de tudo, uma ferramenta de ensino que nasceu de aprender fazendo.

Os arquivos e este relatório estão abertos para quem quiser usar, adaptar e melhorar.

2. Como isso pode mudar a vida das pessoas

Não prometemos milagres. Dizemos o que acreditamos que o projeto pode fazer e o que ainda precisa ser comprovado.

- Derrubar o medo do primeiro contato. Quem joga xadrez já raciocina com regras, condições e consequências, e vê que programar é parecido.
- Aproximar a tecnologia de quem não é da computação. Lógica é a base de planilhas, de automação e de controle de processos, comuns em áreas como a de alimentos.
- Dar ao professor um material pronto, gratuito e reutilizável. Um professor de qualquer cidade pode usar, inclusive onde faltam cursos e laboratórios.
- Desenvolver pensamento crítico. O xadrez ensina a antecipar consequências, planejar e aprender com o erro, habilidades úteis em qualquer profissão.
- Multiplicar. O aluno que entende pode explicar para o colega, e o professor que gosta pode levar a ideia a outra turma.
- Mudar a visão sobre si mesmo. Terminar um desafio de código dá a sensação de eu consigo, que costuma abrir caminho para estudar mais e buscar oportunidades.

O que ainda não sabemos: se esse efeito realmente acontece e em que tamanho. Por isso a metodologia inclui indicadores para medi-lo (seção 4.2).

3. Fundamentação

O projeto se apoia em três ideias da educação em computação:

- Pensamento computacional (Wing, 2006): resolver problemas por decomposição, reconhecimento de padrões, abstração e algoritmos.
- Usar, modificar, criar (Lee et al., 2011): o aluno começa usando um programa pronto, depois o modifica e por fim cria algo próprio.
- Construcionismo (Papert, 1980): aprende-se melhor construindo algo concreto e compartilhável.

Pilar	Onde aparece no projeto
Decomposição	O xeque-mate é dividido em <code>encontrar_rei</code> , <code>casa_atacada</code> , <code>rei_em_xeque</code> , <code>existe_jogada_legal</code> .
Padrões	Um único laço (<code>caminho_livre</code>) serve a Torre, Bispo e Dama. O laço <code>para</code> aninhado se repete em todo o código.
Abstração	<code>dono</code> e <code>peca</code> representam o tabuleiro inteiro com duas matrizes. Uma função <code>logico</code> esconde as regras de cada peça.
Algoritmos	Laço principal, simulação e desfazer, busca de jogada legal, temporizador.

4. Metodologia: Jogada a Jogada

A metodologia propõe uma sequência de sete etapas, no ritmo de uma partida: começa por jogar, passa por entender e construir e termina em compartilhar. Cada etapa tem um papel para o aluno e outro para o professor.

Etapas	O aluno	O professor	Resultado esperado	Tempo
1. Jogar	Joga uma partida com o programa pronto	Apresenta e só observa	Curiosidade e vínculo	20 min

Etapas	O aluno	O professor	Resultado esperado	Tempo
2. Observar	Lista o que a tela faz (botões, sons, mensagens)	Pergunta como cada coisa funciona	Reconhecer comportamentos	15 min
3. Decompor	Associa cada comportamento a uma função do código	Usa o mapa do material didático	Pensar em partes	30 min
4. Construir	Refaz a janela e o tabuleiro (exemplos 1 e 2)	Acompanha em duplas	Criar tela e usar bibliotecas	60 min
5. Testar e depurar	Roda, erra, lê a mensagem e corrige	Incentiva a investigar, sem dar a resposta	Autonomia diante do erro	30 min
6. Remixar	Resolve um desafio no Código_Limpo	Oferece desafios por nível	Aplicar o conteúdo a um problema novo	60 min
7. Compartilhar e refletir	Apresenta o que mudou e explica por quê	Fecha com perguntas de pensamento crítico	Comunicar e consolidar	30 min

4.1 Princípios

- Primeiro o produto, depois o código: a motivação vem do jogo funcionando.
- Versão comentada para o professor, versão limpa para o aluno: quem exercita precisa ler e entender, não só copiar.
- Erro faz parte: o plano de testes e a depuração são etapas oficiais.
- Duplas sempre que possível, para quem sabe mais ensinar quem sabe menos.

4.2 Como medir se funcionou

Indicador	Como coletar	Sinal de sucesso
Confiança para programar	Questionário de 1 a 5 antes e depois	Média maior depois
Compreensão de lógica	5 questões iguais antes e depois	Mais acertos depois
Conclusão do desafio básico	Contagem na aula	A maioria da turma conclui
Autonomia	Quantas dúvidas foram resolvidas sem ajuda	Cresce entre a etapa 4 e a 6
Percepção dos alunos	Perguntas abertas (o que mais gostou? o que mudaria?)	Respostas apontam o jogo como ajuda

Nenhum desses indicadores foi aplicado ainda. Eles formam o plano para a primeira turma.

5. Etapas de desenvolvimento e explicação das funções

Esta é a ordem lógica de construção do jogo. Para cada função, o relatório mostra a imagem do código e responde a três perguntas: o que faz, como foi feita e por que foi feita assim e não de outro jeito.

Etapas 1 Ideia, público e planejamento

Partimos de uma pergunta: como mostrar a alunos que não são de computação que a lógica de programação serve para algo que eles reconhecem? Escolhemos um jogo, e o xadrez venceu porque todo mundo conhece e porque exige planejamento e tática. Levantamos as regras que o programa precisava cobrir e definimos o que ficaria de fora (roque, en passant, computador como adversário), para que o projeto coubesse em um código que um aluno consiga ler.

Etapa 2 Modelagem dos dados

Antes de qualquer desenho, decidimos como o jogo guarda o que acontece. Tudo nasce de duas matrizes 8×8: uma diz quem ocupa a casa e a outra, qual peça está ali. O resto do programa é consequência dessa escolha.

declarações globais

```
declarações globais · linhas 1-56 · parte 1/2

1 programa
2 {
3     // Inclusão das bibliotecas padrão do Portugol Studio
4     inclui biblioteca Graficos --> g
5     inclui biblioteca Mouse --> m
6     inclui biblioteca Util --> u
7     inclui biblioteca Sons --> s
8
9     // Matrizes principais do tabuleiro
10    inteiro dono[8][8] // 0 = vazio, 1 = Jogador 1 (Brancas), 2 = Jogador 2 (Pretas)
11    cadeia peca[8][8] // "P" = Peão, "T" = Torre, "C" = Cavalo, "B" = Bispo, "Q" = Dama, "K" = Rei
12
13    // Variáveis de controle para o histórico de jogadas (funcionalidade "Voltar")
14    inteiro historico_dono[50][64]
15    cadeia historico_pecas[50][64]
16    inteiro historico_turno[50]
17    inteiro quantidade_historico = 0
18
19    // Variáveis de controle geral da partida
20    inteiro som_movimento = -1
21    inteiro turno = 1
22
23    logico encerrado = falso
24    logico selecionada = falso
25    logico arrastando = falso
26    logico trilhas = falso
27    logico tela_cheia = falso
28
29    // Coordenadas e posições auxiliares
30    inteiro sel_linha = -1
31    inteiro sel_coluna = -1
32    inteiro origem_linha = -1
33    inteiro origem_coluna = -1
34    inteiro mouse_x = 0
35    inteiro mouse_y = 0
36
37    // Estados de xeque e xeque-mate
38    logico jogador1_em_xeque = falso
39    logico jogador2_em_xeque = falso
40    logico jogador1_mate = falso
41    logico jogador2_mate = falso
42
43    // Controles para empate e desistência
44    inteiro proposta_empate = 0
45    inteiro solicitacao_desistencia = 0
46    inteiro tempo_inicio_desistencia = 0
47
48    cadeia mensagem = ""
49
50    // Configurações de layout e tamanho da tela
```

```
declarações globais · linhas 1-56 · parte 2/2

51     inteiro tamanho_casa = 70
52     inteiro margem_x = 30
53     inteiro margem_y = 25
54     inteiro botao_y = 600
55
56     // Função principal executada ao iniciar o programa
```

O que faz. Guarda o estado do jogo: tabuleiro, turno, histórico e controles.

Como foi feita. Duas matrizes 8×8 (**dono** e **peca**), três estruturas para o histórico e variáveis **logico** que funcionam como interruptores (**arrastando**, **encerrado**).

Por que assim. Matrizes simples e globais são mais fáceis de ensinar do que um tipo composto, e todas as funções enxergam o mesmo estado sem precisar de parâmetros.

inicializar_tabuleiro

```
linhas 81-128

81 // Posiciona todas as peças nas posições iniciais do xadrez
82 funcao inicializar_tabuleiro()
83 {
84     inteiro l, c
85
86     para (l = 0; l < 8; l++)
87     {
88         para (c = 0; c < 8; c++)
89         {
90             dono[l][c] = 0
91             peca[l][c] = " "
92         }
93     }
94
95     // Peças pretas (Linhas 0 e 1)
96     dono[0][0] = 2 peca[0][0] = "T"
97     dono[0][1] = 2 peca[0][1] = "C"
98     dono[0][2] = 2 peca[0][2] = "B"
99     dono[0][3] = 2 peca[0][3] = "Q"
100    dono[0][4] = 2 peca[0][4] = "K"
101    dono[0][5] = 2 peca[0][5] = "B"
102    dono[0][6] = 2 peca[0][6] = "C"
103    dono[0][7] = 2 peca[0][7] = "T"
104
105    para (c = 0; c < 8; c++)
106    {
107        dono[1][c] = 2
108        peca[1][c] = "P"
109    }
110
111    // Peças brancas (Linhas 6 e 7)
112    dono[7][0] = 1 peca[7][0] = "T"
113    dono[7][1] = 1 peca[7][1] = "C"
114    dono[7][2] = 1 peca[7][2] = "B"
115    dono[7][3] = 1 peca[7][3] = "Q"
116    dono[7][4] = 1 peca[7][4] = "K"
117    dono[7][5] = 1 peca[7][5] = "B"
118    dono[7][6] = 1 peca[7][6] = "C"
119    dono[7][7] = 1 peca[7][7] = "T"
120
121    para (c = 0; c < 8; c++)
122    {
123        dono[6][c] = 1
124        peca[6][c] = "P"
125    }
126
127    mensagem = "Jogo iniciado. Vez do Jogador 1 (Brancas)."
128 }
```

O que faz. Limpa o tabuleiro e posiciona as 32 peças.

Como foi feita. Um **para** aninhado zera as 64 casas. As peças de fundo são atribuídas casa a casa e os peões por um **para** de 8 voltas.

Por que assim. Escrever as peças de fundo à mão deixa a posição inicial visível no código. Os peões repetem o mesmo valor, então o laço é a ferramenta certa.

Etapa 3 Tela e layout

Com os dados prontos, construímos a janela e o desenho. Aqui entram as bibliotecas Gráficos e Util, o laço principal e as medidas proporcionais, que permitem a tela cheia.

inicio

```
linhas 56-79

56 // Função principal executada ao iniciar o programa
57 funcao inicio()
58 {
59     // Carrega o arquivo de som do movimento
60     som_movimento = s.carregar_som("Movimento.mp3")
61
62     g.iniciar_modos_grafico(verdadeiro)
63     g.definir_dimensoes_janela(620, 760)
64     g.definir_titulo_janela("Xadrez V4.1 - Círculos e Letras")
65
66     inicializar_tabuleiro()
67     atualizar_estado_jogo()
68     atualizar_layout()
69
70     // Loop principal do jogo
71     enquanto (verdadeiro)
72     {
73         processar_tempo_desistencia()
74         processar_mouse()
75         atualizar_layout()
76         desenhar_tudo()
77         u.aguarde(15)
78     }
79 }
```

O que faz. Ponto de entrada: abre a janela e roda o ciclo do jogo.

Como foi feita. Carrega o som, ativa o modo gráfico, define tamanho e título, prepara o jogo e entra em um **enquanto (verdadeiro)** que processa, desenha e espera 15 ms.

Por que assim. Um laço contínuo com **u.aguarde(15)** (até cerca de 60 quadros por segundo) mantém a tela viva sem sobrecarregar o processador. Desenhando só depois de cada jogada impediria arrastar as peças.

atualizar_layout

```
atualizar_layout · linhas 130-156

130 // Calcula o tamanho das casas proporcionalmente à janela
131 funcao atualizar_layout()
132 {
133     inteiro largura, altura, espaco_y, novo_tamanho
134
135     largura = g.largura_janela()
136     altura = g.altura_janela()
137
138     espaco_y = altura - 180
139     novo_tamanho = espaco_y / 8
140
141     se ((novo_tamanho * 8) > largura - 40)
142     {
143         novo_tamanho = (largura - 40) / 8
144     }
145
146     se (novo_tamanho < 40) { novo_tamanho = 40 }
147
148     tamanho_casa = novo_tamanho
149     margem_x = (largura - (8 * tamanho_casa)) / 2
150     se (margem_x < 10) { margem_x = 10 }
151
152     margem_y = (espaco_y - (8 * tamanho_casa)) / 2
153     se (margem_y < 10) { margem_y = 10 }
154
155     botao_y = margem_y + (8 * tamanho_casa) + 20
156 }
```

O que faz. Recalcula tamanho das casas e margens conforme a janela.

Como foi feita. Reserva 180 px para botões e mensagens, usa o menor valor entre o que cabe na altura e na largura, impõe mínimo de 40 px e centraliza o tabuleiro.

Por que assim. Medidas proporcionais fazem o tabuleiro funcionar na janela normal e na tela cheia. Com valores fixos, ele ficaria pequeno ou cortado.

desenhar_tudo

```
desenhar_tudo · linhas 681-695

681 // Função central de renderização gráfica na tela
682 funcao desenhar_tudo()
683 {
684     g.definir_cor(0x18232F)
685     g.limpar()
686
687     desenhar_tabuleiro()
688     se (trilhas e selecionada) { desenhar_trilhas() }
689     desenhar_pecas()
690     desenhar_botoes()
691     desenhar_status()
692
693     se (solicitacao_desistencia != 0) { desenhar_caixa_desistencia() }
694     g.renderizar()
695 }
```

O que faz. Monta cada quadro, na ordem certa.

Como foi feita. Pinta o fundo, **limpar**, desenha tabuleiro, trilhas (se ativas), peças, botões e status, a caixa de desistência quando houver, e só no fim chama **renderizar**.

Por que assim. Desenhar de trás para frente evita que um elemento cubra outro. Concentrar **limpar** e **renderizar** em um lugar só evita cintilação e deixa claro onde começa e termina cada quadro.

desenhar_tabuleiro

desenhar_tabuleiro · linhas 697–717

```
697     funcao desenhar_tabuleiro()
698     {
699         inteiro l, c, x, y
700         para (l = 0; l < 8; l++)
701         {
702             para (c = 0; c < 8; c++)
703             {
704                 x = margem_x + c * tamanho_casa
705                 y = margem_y + l * tamanho_casa
706
707                 se ((l + c) % 2 == 0) { g.definir_cor(0xECF0F1) }
708                 senao { g.definir_cor(0x7F8C8D) }
709
710                 se (selecionada e l == sel_linha e c == sel_coluna) { g.definir_cor(0xF1C40F) }
711
712                 g.desenhar_retangulo(x, y, tamanho_casa, tamanho_casa, falso, verdadeiro)
713                 g.definir_cor(0x222222)
714                 g.desenhar_retangulo(x, y, tamanho_casa, tamanho_casa, falso, falso)
715             }
716         }
717     }
```

O que faz. Desenha as 64 casas alternadas.

Como foi feita. para aninhado com $x = \text{margem_x} + c * \text{tamanho_casa}$ e $(l + c) \% 2$ para alternar claro e escuro. A casa selecionada fica amarela e a borda vem de um retângulo vazado.

Por que assim. O resto da divisão dispensa uma tabela de cores e funciona em qualquer tamanho. Também dá aos alunos uma aplicação concreta do operador %.

desenhar_peca

desenhar_peca · linhas 760–782

```
760     // Desenha o círculo base e a letra identificadora da peça
761     funcao desenhar_peca(inteiro x, inteiro y, inteiro jogador, cadeia letra)
762     {
763         inteiro cx, cy, raio
764
765         cx = x + tamanho_casa / 2
766         cy = y + tamanho_casa / 2
767         raio = (tamanho_casa * 75) / 200
768
769         se (jogador == 1) { g.definir_cor(0xFFFFF) }
770         senao { g.definir_cor(0x222222) }
771         g.desenhar_elipse(cx - raio, cy - raio, raio * 2, raio * 2, verdadeiro)
772
773         se (jogador == 1) { g.definir_cor(0x333333) }
774         senao { g.definir_cor(0xCCCCC) }
775         g.desenhar_elipse(cx - raio, cy - raio, raio * 2, raio * 2, falso)
776
777         se (jogador == 1) { g.definir_cor(0x111111) }
778         senao { g.definir_cor(0xFFFFF) }
779
780         g.definir_tamanho_texto((tamanho_casa * 45) / 100)
781         g.desenhar_texto(cx - (tamanho_casa * 15) / 100, cy - (tamanho_casa * 32) / 100, letra)
782     }
```

O que faz. Desenha uma peça: círculo e letra.

Como foi feita. Calcula o centro da casa, o raio proporcional (37,5% do lado), pinta o círculo conforme o jogador, traça o contorno e escreve a letra com cor de contraste.

Por que assim. Círculo e letra dependem só de `desenhar_elipse` e `desenhar_texto`. Sem arquivos de imagem, o jogo não tem dependências e os alunos reproduzem tudo no Portugol.

desenhar_pecas

```
desenhar_pecas · linhas 739-758

739  funcao desenhar_pecas()
740  {
741      inteiro l, c, x, y
742      para (l = 0; l < 8; l++)
743      {
744          para (c = 0; c < 8; c++)
745          {
746              se (dono[l][c] != 0)
747              {
748                  se (nao (arrastando e l == origem_linha e c == origem_coluna))
749                  {
750                      x = margem_x + c * tamanho_casa
751                      y = margem_y + l * tamanho_casa
752                      desenhar_pecas(x, y, dono[l][c], peca[l][c])
753                  }
754              }
755          }
756      }
757      se (arrastando) { desenhar_pecas_arrastada() }
758  }
```

O que faz. Percorre o tabuleiro e desenha todas as peças.

Como foi feita. Varre as 64 casas, desenha onde `dono != 0`, pula a peça que está sendo arrastada e, no fim, desenha essa peça sob o mouse.

Por que assim. Pular a peça de origem evita duplicá-la na tela, e desenhá-la por último a mantém por cima das outras.

desenhar_pecas_arrastada

```
desenhar_pecas_arrastada · linhas 784-790

784  funcao desenhar_pecas_arrastada()
785  {
786      se (origem_linha < 0 ou origem_linha >= 8 ou origem_coluna < 0 ou origem_coluna >= 8) {
787          retorne }
788      inteiro x = mouse_x - tamanho_casa / 2
789      inteiro y = mouse_y - tamanho_casa / 2
790      desenhar_pecas(x, y, dono[origem_linha][origem_coluna], peca[origem_linha][origem_coluna])
791  }
```

O que faz. Desenha a peça presa ao cursor.

Como foi feita. Centraliza a peça nas coordenadas do mouse (`mouse_x - tamanho_casa / 2`), depois de conferir que a origem é válida.

Por que assim. Reaproveita `desenhar_pecas` em vez de repetir código, e a checagem de limites evita erro se o estado ficar inconsistente.

Etapa 4 Interação com o mouse

O jogo precisava responder ao jogador. Usamos a biblioteca Mouse para ler a posição e o botão e criamos o arrastar-e-soltar, que parece mais com um tabuleiro de verdade do que clicar duas vezes.

processar_mouse

processar_mouse · linhas 158–297 · parte 1/3

```
158 // Gerencia cliques e o ato de arrastar peças com o mouse
159 funcao processar_mouse()
160 {
161     inteiro x, y, linha, coluna
162
163     x = m.posicao_x()
164     y = m.posicao_y()
165     mouse_x = x
166     mouse_y = y
167
168     se (m.botao_pressionado(m.BOTAO_ESQUERDO))
169     {
170         // Tratamento caso haja uma solicitação de desistência ativa na tela
171         se (solicitacao_desistencia != 0)
172         {
173             se (x >= 210 e x <= 300 e y >= 360 e y <= 400)
174             {
175                 efetivar_desistencia()
176                 u.aguarde(300)
177                 retorne
178             }
179             se (x >= 320 e x <= 410 e y >= 360 e y <= 400)
180             {
181                 solicitacao_desistencia = 0
182                 mensagem = "Desistência recusada."
183                 u.aguarde(300)
184                 retorne
185             }
186             retorne
187         }
188
189         se (nao arrastando)
190         {
191             // Botão: Voltar jogada
192             se (x >= margem_x e x <= margem_x + 115 e y >= botao_y e y <= botao_y + 40)
193             {
194                 voltar_jogada()
195                 u.aguarde(250)
196                 retorne
197             }
198
199             // Botão: Trilhas
200             se (x >= margem_x + 125 e x <= margem_x + 245 e y >= botao_y e y <= botao_y + 40)
201             {
202                 trilhas = nao trilhas
203                 se (trilhas)
204                 {
205                     mensagem = "Trilhas ativadas."
206                 }
207                 senao
```

```
208         {
209             mensagem = "Trilhas desativadas."
210         }
211         u.aguarde(250)
212         retorne
213     }
214
215     // Botão: Empate
216     se (x >= margem_x + 255 e x <= margem_x + 370 e y >= botao_y e y <= botao_y + 40)
217     {
218         processar_empate()
219         u.aguarde(300)
220         retorne
221     }
222
223     // Botão: Desistir
224     se (x >= margem_x + 380 e x <= margem_x + 520 e y >= botao_y e y <= botao_y + 40)
225     {
226         pedir_desistencia()
227         u.aguarde(300)
228         retorne
229     }
230
231     // Botão: Tela Cheia
232     se (x >= margem_x e x <= margem_x + 150 e y >= botao_y + 50 e y <= botao_y + 90)
233     {
234         alternar_tela_cheia()
235         u.aguarde(400)
236         retorne
237     }
238
239     se (encerrado)
240     {
241         mensagem = "Partida encerrada."
242         retorne
243     }
244
245     linha = (y - margem_y) / tamanho_casa
246     coluna = (x - margem_x) / tamanho_casa
247
248     // Seleciona a peça se pertencer ao jogador da vez
249     se (linha >= 0 e linha < 8 e coluna >= 0 e coluna < 8)
250     {
251         se (dono[linha][coluna] == turno)
252         {
253             arrastando = verdadeiro
254             selecionada = verdadeiro
255             origem_linha = linha
256             origem_coluna = coluna
257             sel_linha = linha
```

```

processar_mouse · linhas 158-297 · parte 3/3

258         sel_coluna = coluna
259         mensagem = "Arrastando peça..."
260     }
261 }
262 }
263     retorne
264 }
265
266 // Ao soltar o botão do mouse, valida se o movimento é legal
267 se (arrastando)
268 {
269     linha = (mouse_y - margem_y) / tamanho_casa
270     coluna = (mouse_x - margem_x) / tamanho_casa
271
272     se (linha < 0 ou linha >= 8 ou coluna < 0 ou coluna >= 8 ou (linha == origem_linha e
273         coluna == origem_coluna))
274     {
275         cancelar_arrasto()
276         mensagem = "Movimento cancelado."
277         retorne
278     }
279
280     se (movimento_valido(origem_linha, origem_coluna, linha, coluna))
281     {
282         se (movimento_deixa_rei_em_xeque(origem_linha, origem_coluna, linha, coluna, turno))
283         {
284             mensagem = "Jogada proibida: Rei em xeque."
285         }
286         senao
287         {
288             realizar_movimento(origem_linha, origem_coluna, linha, coluna)
289         }
290     }
291     senao
292     {
293         mensagem = "Movimento inválido."
294     }
295     cancelar_arrasto()
296 }
297 }

```

O que faz. Entende cliques, botões e o arrastar-e-soltar.

Como foi feita. Lê posição e botão. Com o botão pressionado trata, nesta ordem, a caixa de desistência, os botões da interface e a seleção da peça ($linha = (y - margem_y) / tamanho_casa$). Ao soltar, converte o ponto em casa, valida e executa.

Por que assim. A divisão inteira transforma pixels em linha e coluna sem tabelas. Arrastar é mais natural do que clicar duas vezes. Os u.aguarde depois de cada botão evitam cliques repetidos.

cancelar_arrasto

```

cancelar_arrasto · linhas 299-307

299     funcao cancelar_arrasto()
300     {
301         arrastando = falso
302         selecionada = falso
303         origem_linha = -1
304         origem_coluna = -1
305         sel_linha = -1
306         sel_coluna = -1
307     }

```

O que faz. Limpa a seleção.

Como foi feita. Volta **arrastando** e **selecionada** para **falso** e as coordenadas para -1.

Por que assim. Vários pontos precisam dessa limpeza (jogada feita, cancelada, desistência). Uma função evita repetir e esquecer algum campo.

Etapa 5 Regras de movimento

Cada peça se move de um jeito. Transformamos as regras em condições e em uma função de caminho livre, e escrevemos a função que executa a jogada.

movimento_valido

```
● ● ● movimento_valido · linhas 345–383

345 // Valida as regras de movimento geométricas de cada peça de xadrez
346 funcao logico movimento_valido(inteiro origem_l, inteiro origem_c, inteiro destino_l, inteiro
    destino_c)
347 {
348     cadeia tipo
349     inteiro dif_l, dif_c, abs_l, abs_c, direcao
350
351     tipo = peca[origem_l][origem_c]
352     se (dono[destino_l][destino_c] == turno) { retorne falso }
353
354     dif_l = destino_l - origem_l
355     dif_c = destino_c - origem_c
356
357     se (dif_l < 0) { abs_l = -dif_l } senao { abs_l = dif_l }
358     se (dif_c < 0) { abs_c = -dif_c } senao { abs_c = dif_c }
359
360     // Regras do Peão
361     se (tipo == "P")
362     {
363         se (turno == 2) { direcao = 1 } senao { direcao = -1 }
364
365         se (destino_c == origem_c e destino_l == origem_l + direcao e dono[destino_l][destino_c]
            == 0) { retorne verdadeiro }
366         se (destino_c == origem_c e dono[destino_l][destino_c] == 0)
367         {
368             se (turno == 1 e origem_l == 6 e destino_l == 4 e dono[5][origem_c] == 0) { retorne
                verdadeiro }
369             se (turno == 2 e origem_l == 1 e destino_l == 3 e dono[2][origem_c] == 0) { retorne
                verdadeiro }
370         }
371         se (abs_c == 1 e destino_l == origem_l + direcao e dono[destino_l][destino_c] != 0) {
            retorne verdadeiro }
372         retorne falso
373     }
374
375     // Regras do Rei, Cavalo, Torre, Bispo e Dama
376     se (tipo == "K" e abs_l <= 1 e abs_c <= 1) { retorne verdadeiro }
377     se (tipo == "C" e ((abs_l == 2 e abs_c == 1) ou (abs_l == 1 e abs_c == 2))) { retorne
        verdadeiro }
378     se (tipo == "T" e (dif_l == 0 ou dif_c == 0)) { retorne caminho_livre(origem_l, origem_c,
        destino_l, destino_c) }
379     se (tipo == "B" e abs_l == abs_c) { retorne caminho_livre(origem_l, origem_c, destino_l,
        destino_c) }
380     se (tipo == "Q" e (dif_l == 0 ou dif_c == 0 ou abs_l == abs_c)) { retorne caminho_livre(
        origem_l, origem_c, destino_l, destino_c) }
381
382     retorne falso
383 }
```

O que faz. Diz se a geometria do movimento é permitida.

Como foi feita. Calcula diferenças e valores absolutos. Trata o peão à parte (frente, saída de 2 casas, captura na diagonal) e depois Rei, Cavalo, Torre, Bispo e Dama com **se**.

Por que assim. Retornar **logico** permite reaproveitar a mesma regra para jogar, mostrar trilhas, achar xeque e procurar jogadas. O valor absoluto é feito à mão com **se**, o que ensina o conceito e dispensa outra biblioteca.

caminho_livre

```
● ● ● caminho_livre · Linhas 385–405

385 // Verifica se há peças bloqueando o caminho de Torres, Bispos ou Damas
386 funcao logico caminho_livre(inteiro origem_l, inteiro origem_c, inteiro destino_l, inteiro
    destino_c)
387 {
388     inteiro passo_l = 0, passo_c = 0, l, c
389
390     se (destino_l > origem_l) { passo_l = 1 }
391     se (destino_l < origem_l) { passo_l = -1 }
392     se (destino_c > origem_c) { passo_c = 1 }
393     se (destino_c < origem_c) { passo_c = -1 }
394
395     l = origem_l + passo_l
396     c = origem_c + passo_c
397
398     enquanto (l != destino_l ou c != destino_c)
399     {
400         se (dono[l][c] != 0) { retorne falso }
401         l = l + passo_l
402         c = c + passo_c
403     }
404     retorne verdadeiro
405 }
```

O que faz. Verifica se há peça no caminho de Torre, Bispo ou Dama.

Como foi feita. Descobre o passo (-1, 0 ou 1) em linha e coluna e anda casa a casa até o destino, parando se achar uma peça.

Por que assim. Reta e diagonal são o mesmo caminho com passos diferentes, então um laço só serve às três peças. O Cavalo pula e o Rei anda uma casa, por isso não precisam dele.

realizar_movimento

```
● ● ● realizar_movimento · linhas 309-343

309 // Executa a movimentação oficial e troca o turno
310 funcao realizar_movimento(inteiro origem_l, inteiro origem_c, inteiro destino_l, inteiro
    destino_c)
311 {
312     guardar_historico()
313
314     dono[destino_l][destino_c] = dono[origem_l][origem_c]
315     peca[destino_l][destino_c] = peca[origem_l][origem_c]
316
317     dono[origem_l][origem_c] = 0
318     peca[origem_l][origem_c] = " "
319
320     s.reproduzir_som(som_movimento, falso)
321
322     // Promoção automática do peão ao chegar no fim do tabuleiro
323     se (peca[destino_l][destino_c] == "P" e (destino_l == 0 ou destino_l == 7))
324     {
325         peca[destino_l][destino_c] = "Q"
326         mensagem = "Peão promovido a Dama."
327     }
328     senao
329     {
330         mensagem = "Jogada realizada."
331     }
332
333     se (turno == 1)
334     {
335         turno = 2
336     }
337     senao
338     {
339         turno = 1
340     }
341
342     atualizar_estado_jogo()
343 }
```

O que faz. Executa a jogada e passa a vez.

Como foi feita. Salva o histórico, copia peça e dono para o destino, esvazia a origem, toca o som, promove o peão a Dama no fim do tabuleiro, troca o turno e reavalia o estado.

Por que assim. Salvar o histórico antes de mexer permite desfazer. A promoção automática a Dama (a escolha mais comum) evita criar um menu extra nesta versão.

Etapa 6 Xeque e xeque-mate

É a parte mais difícil e a mais interessante. A solução foi dividir o problema em perguntas pequenas e encaixá-las: achar o rei, saber se uma casa é atacada, simular a jogada e procurar uma saída.

encontrar_rei_linha

```
encontrar_rei_linha · linhas 407-417

407     funcao inteiro encontrar_rei_linha(inteiro jogador)
408     {
409         para (inteiro l = 0; l < 8; l++)
410         {
411             para (inteiro c = 0; c < 8; c++)
412             {
413                 se (dono[l][c] == jogador e peca[l][c] == "K") { retorne l }
414             }
415         }
416         retorne -1
417     }
```

O que faz. Localiza a linha do rei de um jogador.

Como foi feita. Percorre a matriz procurando "K" do jogador e devolve a linha, ou -1 se não achar.

Por que assim. Como uma função devolve um valor só, separamos linha e coluna. É simples e mostra bem o conceito de retorno.

encontrar_rei_coluna

```
encontrar_rei_coluna · linhas 419-429

419     funcao inteiro encontrar_rei_coluna(inteiro jogador)
420     {
421         para (inteiro l = 0; l < 8; l++)
422         {
423             para (inteiro c = 0; c < 8; c++)
424             {
425                 se (dono[l][c] == jogador e peca[l][c] == "K") { retorne c }
426             }
427         }
428         retorne -1
429     }
```

O que faz. Localiza a coluna do rei de um jogador.

Como foi feita. Mesma busca da função anterior, devolvendo a coluna.

Por que assim. Manter as duas funções separadas deixa cada uma com uma única responsabilidade.

casa_atacada

```
● ● ● casa_atacada · linhas 431-464

431 // Checa se uma casa específica está sob ataque inimigo
432 funcao logico casa_atacada(inteiro alvo_l, inteiro alvo_c, inteiro atacante)
433 {
434     inteiro l, c, dif_l, dif_c, abs_l, abs_c
435     cadeia tipo
436
437     para (l = 0; l < 8; l++)
438     {
439         para (c = 0; c < 8; c++)
440         {
441             se (dono[l][c] == atacante)
442             {
443                 tipo = peca[l][c]
444                 dif_l = alvo_l - l
445                 dif_c = alvo_c - c
446
447                 se (dif_l < 0) { abs_l = -dif_l } senao { abs_l = dif_l }
448                 se (dif_c < 0) { abs_c = -dif_c } senao { abs_c = dif_c }
449
450                 se (tipo == "P")
451                 {
452                     se (atacante == 1 e alvo_l == l - 1 e abs_c == 1) { retorne verdadeiro }
453                     se (atacante == 2 e alvo_l == l + 1 e abs_c == 1) { retorne verdadeiro }
454                 }
455                 senao se (tipo == "C" e ((abs_l == 2 e abs_c == 1) ou (abs_l == 1 e abs_c == 2))
456                     ) { retorne verdadeiro }
457                 senao se (tipo == "K" e abs_l <= 1 e abs_c <= 1) { retorne verdadeiro }
458                 senao se (tipo == "T" e (dif_l == 0 ou dif_c == 0) e caminho_livre(l, c, alvo_l,
459                     alvo_c)) { retorne verdadeiro }
460                 senao se (tipo == "B" e abs_l == abs_c e caminho_livre(l, c, alvo_l, alvo_c)) {
461                     retorne verdadeiro }
462                 senao se (tipo == "Q" e (dif_l == 0 ou dif_c == 0 ou abs_l == abs_c) e
463                     caminho_livre(l, c, alvo_l, alvo_c)) { retorne verdadeiro }
464             }
465         }
466     }
467     retorne falso
468 }
```

O que faz. Responde se uma casa está sob ataque de um jogador.

Como foi feita. Percorre todas as peças do atacante e aplica a regra de captura de cada tipo, usando `caminho_livre` para as peças de longo alcance.

Por que assim. O peão ataca na diagonal mesmo com a casa vazia, por isso ele tem um trecho próprio e não usa `movimento_valido`. Para as demais peças, a regra de ataque é a mesma do movimento.

rei_em_xeque

```
● ● ● rei_em_xeque · linhas 466-477

466 funcao logico rei_em_xeque(inteiro jogador)
467 {
468     inteiro linha, coluna, adversario
469     linha = encontrar_rei_linha(jogador)
470     coluna = encontrar_rei_coluna(jogador)
471
472     se (linha == -1 ou coluna == -1) { retorne verdadeiro }
473
474     se (jogador == 2) { adversario = 1 } senao { adversario = 2 }
475
476     retorne casa_atacada(linha, coluna, adversario)
477 }
```

O que faz. Diz se o rei de um jogador está em xeque.

Como foi feita. Acha o rei, define o adversário e pergunta a `casa_atacada`. Rei ausente conta como xeque.

Por que assim. É decomposição de problemas: onde está o rei? aquela casa é atacada? Cada pergunta é uma função pequena.

`movimento_deixa_rei_em_xeque`

```
● ● ● movimento_deixa_rei_em_xeque · linhas 479-504

479 // Simula um movimento para garantir que ele não deixará o próprio rei em xeque
480 funcao logico movimento_deixa_rei_em_xeque(inteiro origem_l, inteiro origem_c, inteiro destino_l
    , inteiro destino_c, inteiro jogador)
481 {
482     inteiro dono_origem, dono_destino
483     cadeia peca_origem, peca_destino
484     logico resultado
485
486     dono_origem = dono[origem_l][origem_c]
487     peca_origem = peca[origem_l][origem_c]
488     dono_destino = dono[destino_l][destino_c]
489     peca_destino = peca[destino_l][destino_c]
490
491     dono[destino_l][destino_c] = dono_origem
492     peca[destino_l][destino_c] = peca_origem
493     dono[origem_l][origem_c] = 0
494     peca[origem_l][origem_c] = " "
495
496     resultado = rei_em_xeque(jogador)
497
498     dono[origem_l][origem_c] = dono_origem
499     peca[origem_l][origem_c] = peca_origem
500     dono[destino_l][destino_c] = dono_destino
501     peca[destino_l][destino_c] = peca_destino
502
503     retorne resultado
504 }
```

O que faz. Impede jogadas que deixam o próprio rei em perigo.

Como foi feita. Guarda o conteúdo das duas casas, faz a jogada no tabuleiro, pergunta `rei_em_xeque`, devolve tudo ao lugar e retorna a resposta.

Por que assim. Simular e desfazer reaproveita as funções prontas. A alternativa, uma regra especial para cada tipo de peça, seria enorme e cheia de exceções.

existe_jogada_legal

● ● ● existe_jogada_legal · linhas 506–536

```
506     funcao logico existe_jogada_legal(inteiro jogador)
507     {
508         inteiro l1, c1, l2, c2, turno_antigo
509
510         turno_antigo = turno
511         turno = jogador
512
513         para (l1 = 0; l1 < 8; l1++)
514         {
515             para (c1 = 0; c1 < 8; c1++)
516             {
517                 se (dono[l1][c1] == jogador)
518                 {
519                     para (l2 = 0; l2 < 8; l2++)
520                     {
521                         para (c2 = 0; c2 < 8; c2++)
522                         {
523                             se (movimento_valido(l1, c1, l2, c2) e nao movimento_deixa_rei_em_xeque(
524                                 l1, c1, l2, c2, jogador))
525                             {
526                                 turno = turno_antigo
527                                 retorne verdadeiro
528                             }
529                         }
530                     }
531                 }
532             }
533
534             turno = turno_antigo
535             retorne falso
536         }
```

O que faz. Procura qualquer jogada legal para um jogador.

Como foi feita. Testa cada peça do jogador contra as 64 casas com `movimento_valido` e `movimento_deixa_rei_em_xeque`, trocando `turno` por um instante.

Por que assim. É força bruta: no máximo cerca de mil verificações, o que é rápido num tabuleiro 8×8 e muito mais simples que um gerador de jogadas. O `turno` volta ao valor original ao sair.

atualizar_estado_jogo

```
● ● ● atualizar_estado_jogo · linhas 538-567

538 // Atualiza o estado atual da partida (verificando se houve xeque ou xeque-mate)
539 funcao atualizar_estado_jogo()
540 {
541     jogador1_em_xeque = rei_em_xeque(1)
542     jogador2_em_xeque = rei_em_xeque(2)
543     jogador1_mate = falso
544     jogador2_mate = falso
545
546     se (turno == 1 e jogador1_em_xeque)
547     {
548         se (nao existe_jogada_legal(1))
549         {
550             jogador1_mate = verdadeiro
551             encerrado = verdadeiro
552             mensagem = "XEQUE-MATE! JOGADOR 2 VENCEU!"
553         }
554         senao { mensagem = "XEQUE! Rei em perigo." }
555     }
556
557     se (turno == 2 e jogador2_em_xeque)
558     {
559         se (nao existe_jogada_legal(2))
560         {
561             jogador2_mate = verdadeiro
562             encerrado = verdadeiro
563             mensagem = "XEQUE-MATE! JOGADOR 1 VENCEU!"
564         }
565         senao { mensagem = "XEQUE! Rei em perigo." }
566     }
567 }
```

O que faz. Atualiza xeque e xeque-mate depois de cada jogada.

Como foi feita. Recalcula `jogador1_em_xeque` e `jogador2_em_xeque`. Se o jogador da vez está em xeque e `existe_jogada_legal` é falso, marca o mate e encerra. Senão, avisa o xeque.

Por que assim. Xeque-mate é xeque somado a nenhuma jogada legal. Escrever o código do jeito que o xadrez define a regra torna fácil explicá-lo.

Etapas 7 Recursos da partida

Funções que tornam o jogo mais justo e mais agradável: voltar jogada, empate por acordo e desistência com confirmação e tempo limite.

guardar_historico

```
guardar_historico · linhas 569-587

569 // Salva o estado atual na matriz de histórico
570 funcao guardar_historico()
571 {
572     inteiro l, c, posicao
573     se (quantidade_historico >= 50) { retorne }
574
575     para (l = 0; l < 8; l++)
576     {
577         para (c = 0; c < 8; c++)
578         {
579             posicao = l * 8 + c
580             historico_dono[quantidade_historico][posicao] = dono[l][c]
581             historico_peca[quantidade_historico][posicao] = peca[l][c]
582         }
583     }
584
585     historico_turno[quantidade_historico] = turno
586     quantidade_historico++
587 }
```

O que faz. Tira uma foto do tabuleiro antes da jogada.

Como foi feita. Copia **dono** e **peca** para a linha **quantidade_historico** das matrizes de histórico, usando **posicao = l * 8 + c**, e guarda o turno. O limite é de 50 jogadas.

Por que assim. Guardar o estado inteiro é mais simples e seguro do que registrar só o movimento, porque desfazer capturas e promoções exigiria lembrar o que foi capturado. A fórmula **l * 8 + c** transforma duas dimensões em uma.

voltar_jogada

```
voltar_jogada · linhas 589-618

589 // Desfaz a última jogada realizada
590 funcao voltar_jogada()
591 {
592     inteiro l, c, posicao
593     se (quantidade_historico <= 0)
594     {
595         mensagem = "Impossível voltar jogada."
596         retorne
597     }
598
599     quantidade_historico--
600
601     para (l = 0; l < 8; l++)
602     {
603         para (c = 0; c < 8; c++)
604         {
605             posicao = l * 8 + c
606             dono[l][c] = historico_dono[quantidade_historico][posicao]
607             peca[l][c] = historico_peca[quantidade_historico][posicao]
608         }
609     }
610
611     turno = historico_turno[quantidade_historico]
612     encerrado = falso
613     proposta_empate = 0
614     solicitacao_desistencia = 0
615     cancelar_arrasto()
616     atualizar_estado_jogo()
617     mensagem = "Jogada desfeita."
618 }
```

O que faz. Desfaz a última jogada.

Como foi feita. Diminui o contador, copia a foto de volta, restaura o turno, cancela empate, desistência e arrasto, reabre a partida e reavalia o xeque.

Por que assim. Restaurar tudo de uma vez evita estados contraditórios, como partida encerrada depois de voltar. Cada variável de controle precisa ser reiniciada, o que também ensina cuidado com o estado.

processar_empate

```
processar_empate · Linhas 620-634

620     funcao processar_empate()
621     {
622         se (encerrado) { retorne }
623         se (proposta_empate == 0)
624         {
625             proposta_empate = turno
626             mensagem = "Empate proposto. O oponente deve aceitar."
627             retorne
628         }
629         se (proposta_empate == turno) { retorne }
630         encerrado = verdadeiro
631         proposta_empate = 0
632         cancelar_arrasto()
633         mensagem = "EMPATE ACORDADO!"
634     }
```

O que faz. Empate por acordo.

Como foi feita. O primeiro clique registra a proposta do jogador da vez. Quando o oponente clica, a partida termina em empate.

Por que assim. Uma variável (`proposta_empate`) guarda quem propôs e basta compará-la com `turno`. É uma máquina de estados mínima, ótima para ensinar o conceito.

pedir_desistencia

```
pedir_desistencia · Linhas 636-642

636     funcao pedir_desistencia()
637     {
638         se (encerrado) { retorne }
639         solicitacao_desistencia = turno
640         tempo_inicio_desistencia = u.tempo_decorrido()
641         mensagem = "Desistência solicitada. Confirme no painel."
642     }
```

O que faz. Abre o pedido de desistência.

Como foi feita. Registra quem pediu e o instante, com `u.tempo_decorrido()`.

Por que assim. Guardar o instante inicial permite medir o tempo depois sem pausar o jogo, e o laço principal continua rodando.

processar_tempo_desistencia

```
processar_tempo_desistencia · linhas 644-655

644     funcao processar_tempo_desistencia()
645     {
646         se (solicitacao_desistencia != 0)
647         {
648             inteiro decorrido = (u.tempo_decorrido() - tempo_inicio_desistencia) / 1000
649             se (decorrido >= 10)
650             {
651                 solicitacao_desistencia = 0
652                 mensagem = "Tempo de desistência expirado."
653             }
654         }
655     }
```

O que faz. Cancela o pedido se passarem 10 segundos.

Como foi feita. É chamada a cada quadro, converte a diferença de tempo em segundos e, ao chegar a 10, zera o pedido.

Por que assim. Verificar a cada volta do laço é a forma natural de ter um temporizador em um programa de uma linha de execução só.

efetivar_desistencia

```
efetivar_desistencia · linhas 657-672

657     funcao efetivar_desistencia()
658     {
659         encerrado = verdadeiro
660         cancelar_arrasto()
661         se (solicitacao_desistencia == 1)
662         {
663             mensagem = "J1 desistiu. J2 VENCEU!"
664             jogador2_mate = verdadeiro
665         }
666         senao
667         {
668             mensagem = "J2 desistiu. J1 VENCEU!"
669             jogador1_mate = verdadeiro
670         }
671         solicitacao_desistencia = 0
672     }
```

O que faz. Confirma a desistência e declara o vencedor.

Como foi feita. Encerra a partida, limpa o arrasto e marca o outro jogador como vencedor, definindo a mensagem.

Por que assim. Separar pedir de efetivar permite que o adversário confirme ou recuse, e deixa cada função com uma responsabilidade.

Etapa 8 Acabamento e interface

Tela cheia, trilhas que mostram as jogadas possíveis, botões, caixa de confirmação e mensagens de status. É o que transforma um programa que funciona em um programa agradável de usar.

alternar_tela_cheia

```
alternar_tela_cheia · linhas 674-679

674     funcao alternar_tela_cheia()
675     {
676         se (tela_cheia) { g.sair_mod0_tela_cheia() tela_cheia = falso }
677         senao { g.entrar_mod0_tela_cheia() tela_cheia = verdadeiro }
678         atualizar_layout()
679     }
```

O que faz. Liga e desliga a tela cheia.

Como foi feita. Se `tela_cheia` é verdadeiro, sai do modo; senão, entra. Inverte a variável e chama `atualizar_layout`.

Por que assim. Uma variável lógica como interruptor é o padrão mais simples. Recalcular o layout logo depois faz o tabuleiro se ajustar.

desenhar_trilhas

```
desenhar_trilhas · linhas 719-737

719     // Desenha as bolinhas verdes de marcação de movimento válido
720     funcao desenhar_trilhas()
721     {
722         inteiro l, c, x, y, raio = tamanho_casa / 4
723
724         para (l = 0; l < 8; l++)
725         {
726             para (c = 0; c < 8; c++)
727             {
728                 se (movimento_valido(sel_linha, sel_coluna, l, c) e nao movimento_deixa_rei_em_xeque
729                     (sel_linha, sel_coluna, l, c, turno))
730                 {
731                     x = margem_x + c * tamanho_casa + (tamanho_casa / 2) - (raio / 2)
732                     y = margem_y + l * tamanho_casa + (tamanho_casa / 2) - (raio / 2)
733                     g.definir_cor(0x27AE60)
734                     g.desenhar_elipse(x, y, raio, raio, verdadeiro)
735                 }
736             }
737         }
```

O que faz. Mostra bolinhas nas casas possíveis.

Como foi feita. Percorre as 64 casas e desenha uma elipse verde nas que passam em `movimento_valido` e em `nao movimento_deixa_rei_em_xeque`.

Por que assim. Reaproveita as regras do jogo, então nunca mostra um movimento que o jogo recusaria. É um apoio para iniciantes, ativável no botão.

desenhar_botoes

```
desenhar_botoes · Linhas 792-821

792 // Desenha os botões de controle na interface
793 funcao desenhar_botoes()
794 {
795     g.definir_cor(0x34495E)
796     g.desenhar_retangulo(margem_x, botao_y, 115, 40, verdadeiro, verdadeiro)
797
798     se (trilhas) { g.definir_cor(0x27AE60) }
799     senao { g.definir_cor(0x34495E) }
800
801     g.desenhar_retangulo(margem_x + 125, botao_y, 120, 40, verdadeiro, verdadeiro)
802     g.definir_cor(0x8E44AD) g.desenhar_retangulo(margem_x + 255, botao_y, 115, 40, verdadeiro,
803         verdadeiro)
804     g.definir_cor(0xC0392B) g.desenhar_retangulo(margem_x + 380, botao_y, 140, 40, verdadeiro,
805         verdadeiro)
806     g.definir_cor(0x16A085) g.desenhar_retangulo(margem_x, botao_y + 50, 150, 40, verdadeiro,
807         verdadeiro)
808
809     g.definir_cor(g.COR_BRANCO)
810     g.definir_tamanho_texto(14.0)
811     g.desenhar_texto(margem_x + 28, botao_y + 25, "VOLTAR")
812     g.desenhar_texto(margem_x + 151, botao_y + 25, "TRILHAS")
813     g.desenhar_texto(margem_x + 278, botao_y + 25, "EMPATE")
814     g.desenhar_texto(margem_x + 416, botao_y + 25, "DESISTIR")
815
816     se (tela_cheia)
817     {
818         g.desenhar_texto(margem_x + 25, botao_y + 75, "SAIR TELA CHEIA")
819     }
820     senao
821     {
822         g.desenhar_texto(margem_x + 25, botao_y + 75, "TELA CHEIA")
823     }
824 }
```

O que faz. Desenha os cinco botões e seus textos.

Como foi feita. Retângulos coloridos a partir de `margem_x` e `botao_y`. O botão Trilhas muda de cor quando ativo e o texto da tela cheia muda conforme o estado.

Por que assim. Um botão é só um retângulo mais uma área de clique em `processar_mouse`. Os alunos veem que botão é uma ideia, não um objeto pronto.

desenhar_caixa_desistencia

```
desenhar_caixa_desistencia · linhas 823-852

823     funcao desenhar_caixa_desistencia()
824     {
825         inteiro decorrido = (u.tempo_decorrido() - tempo_inicio_desistencia) / 1000
826         inteiro restantes = 10 - decorrido
827         se (restantes < 0) { restantes = 0 }
828
829         g.definir_cor(0x111111) g.desenhar_retangulo(130, 270, 360, 160, verdadeiro, verdadeiro)
830         g.definir_cor(0xE74C3C) g.desenhar_retangulo(130, 270, 360, 160, falso, falso)
831
832         g.definir_cor(g.COR_BRANCO)
833         g.definir_tamanho_texto(14.0)
834
835         cadeia texto_des
836         se (solicitacao_desistencia == 1)
837         {
838             texto_des = "J1 quer desistir. J2 aceita?"
839         }
840         senao
841         {
842             texto_des = "J2 quer desistir. J1 aceita?"
843         }
844
845         g.desenhar_texto(150, 300, texto_des + " (" + restantes + "s)")
846
847         g.definir_cor(0x27AE60) g.desenhar_retangulo(210, 360, 90, 40, verdadeiro, verdadeiro)
848         g.definir_cor(g.COR_BRANCO) g.desenhar_texto(240, 385, "SIM")
849
850         g.definir_cor(0xC0392B) g.desenhar_retangulo(320, 360, 90, 40, verdadeiro, verdadeiro)
851         g.definir_cor(g.COR_BRANCO) g.desenhar_texto(350, 385, "NÃO")
852     }
```

O que faz. Mostra a confirmação com contagem regressiva.

Como foi feita. Calcula os segundos restantes, desenha a caixa, o texto com o jogador e os botões SIM e NÃO.

Por que assim. Uma caixa desenhada por último, sobre o resto, deixa claro que o jogo espera uma resposta. Enquanto ela existe, `processar_mouse` ignora o tabuleiro.

desenhar_status

```
desenhar_status · Linhas 854-887

854 // Mostra na tela o status da partida, turnos e mensagens de feedback
855 funcao desenhar_status()
856 {
857     g.definir_tamanho_texto(16.0)
858
859     se (jogador1_mate ou jogador2_mate)
860     {
861         g.definir_cor(0x2ECC71)
862         g.desenhar_texto(margem_x, botao_y + 110, "FIM DE JOGO - XEQUE-MATE!")
863         retorne
864     }
865
866     se (jogador1_em_xeque ou jogador2_em_xeque)
867     {
868         g.definir_cor(0xE67E22)
869         g.desenhar_texto(margem_x, botao_y + 110, "XEQUE! Rei ameaçado.")
870     }
871     senao
872     {
873         g.definir_cor(g.COR_BRANCO)
874         se (turno == 1)
875         {
876             g.desenhar_texto(margem_x, botao_y + 110, "VEZ: JOGADOR 1 (BRANCAS)")
877         }
878         senao
879         {
880             g.desenhar_texto(margem_x, botao_y + 110, "VEZ: JOGADOR 2 (PRETAS)")
881         }
882     }
883
884     g.definir_cor(0xBDC3C7)
885     g.definir_tamanho_texto(14.0)
886     g.desenhar_texto(margem_x, botao_y + 135, mensagem)
887 }
```

O que faz. Mostra a vez, o xeque, o fim de jogo e as mensagens.

Como foi feita. Escolhe cor e texto conforme o estado (mate, xeque, vez de cada jogador) e escreve a **mensagem** logo abaixo.

Por que assim. Feedback constante diz ao jogador o que aconteceu. O **retorne** no caso de mate evita sobrescrever o aviso final.

Etapa 9 · Testes, versões e documentação

O título da janela mostra que o código está na **versão 4.1**, resultado de várias rodadas de teste e ajuste. Ao final, separamos o programa em duas versões: a **Código_Institucional.por**, com comentários, para o professor ensinar, e a **Código_Limpo.por**, sem comentários, para os alunos exercitarem. As duas têm a mesma lógica. O plano de testes da seção 7 serve de roteiro para validar o jogo antes de cada aula.

Etapa 10 · Aplicação em sala e evolução

A última etapa é a que ainda falta: levar o jogo à sala, observar os alunos, medir o que aprenderam (seção 4.2) e usar o que for descoberto para melhorar o código e o material. O projeto foi pensado para continuar crescendo.

6. Decisões de projeto e alternativas

Decisão	Alternativa	Por que escolhemos assim
Duas matrizes (dono , peca)	Uma matriz de números com código para cada peça	Letras como "T" e "C" deixam o código legível. Separar o dono da peça simplifica as regras.
Peças como círculos e letras	Imagens das peças	Sem arquivos externos, o jogo roda em qualquer computador e os alunos reproduzem tudo.
Histórico com cópia do tabuleiro	Guardar só os movimentos	Copiar o estado desfaz também capturas e promoções, sem lógica extra.
Simular e desfazer para validar xequ	Regras específicas por peça	Reaproveita funções prontas e evita exceções.
Laço enquanto com u.aguarde(15)	Redesenhar só após cada jogada	Necessário para arrastar peças e mostrar o contador de desistência.
Promoção automática a Dama	Menu de escolha da peça	Mantém o código curto. Vira desafio para os alunos.
Arrastar com o mouse	Clicar na origem e no destino	É mais natural e mostra mais conceitos (estado de arrasto).
Força bruta em existe_jogada_legal	Gerador de jogadas por peça	No máximo cerca de mil verificações, e o código é bem mais fácil de ensinar.
Estado em variáveis globais	Passar tudo por parâmetros	Menos código para iniciantes. O custo é que as funções dependem do estado global.

7. Plano de testes

Roteiro para validar o jogo antes de cada aula. Todos os testes estão por executar na primeira turma.

Teste	Ação	Resultado esperado
Início	Abrir o programa	Tabuleiro 8×8 com 32 peças e vez do Jogador 1
Peão	Mover 2 casas na primeira jogada e 2 na segunda	Primeira aceita, segunda recusada
Caminho	Mover a Torre sobre outra peça	Movimento inválido
Cavalo	Mover o Cavalo sobre peças	Movimento aceito
Xequ-mate (Mate do Louco)	Brancas f2-f3 e g2-g4, Pretas e7-e5 e Dama d8-h4	"XEQUE-MATE! JOGADOR 2 VENCEU!"
Rei exposto	Mover uma peça que protege o próprio rei	"Jogada proibida: Rei em xeque."
Voltar	Jogar e clicar em VOLTAR	Peças e vez restaurados
Desistência	Pedir e não confirmar por 10 segundos	Pedido expira
Promoção	Levar um peão ao fim do tabuleiro	Peão vira Dama
Tela cheia	Ativar e desativar	Tabuleiro se ajusta

8. Limitações e próximos passos

Em vez de esconder os limites, tratamos como desafios para os alunos:

- Faltam roque, en passant, escolha da peça na promoção e empates por afogamento, repetição ou material insuficiente.
- Não há relógio de partida nem adversário controlado pelo computador.
- As áreas de clique dos botões repetem os números usados no desenho. Se um botão mudar de lugar, é preciso alterar nos dois pontos. Guardar essas posições em variáveis seria uma boa melhoria.
- A caixa de desistência usa posição fixa na tela, enquanto o tabuleiro se adapta.
- O histórico guarda até 50 jogadas.

9. Conclusão

O projeto reúne programação, raciocínio estratégico e pensamento crítico em uma atividade só. Ele mostra que a lógica pode ser aprendida de forma divertida e que uma linguagem feita apenas para ensinar também cria jogos. O próximo passo é levá-lo à sala, medir o que os alunos aprenderam e melhorar tudo com o que descobriremos. Agradecemos ao Professor José Alcimar e ao IFPA pelo espaço.

Referências

- Wing, J. M. Computational thinking. Communications of the ACM, v. 49, n. 3, p. 33-35, 2006.
- Lee, I. et al. Computational thinking for youth in practice. ACM Inroads, v. 2, n. 1, p. 32-37, 2011.
- Papert, S. Mindstorms: children, computers, and powerful ideas. New York: Basic Books, 1980.

Contatos

- GitHub: github.com/Dinneiro
- LinkedIn: [linkedin.com/in/thiago-oliveira-8a2724356](https://www.linkedin.com/in/thiago-oliveira-8a2724356)
- E-mails: thiagosilvafx5@gmail.com e allanrasec2018@gmail.com